C++ 原子操作和内存模型

Posted on 2023-10-19 15:57:38 by 恋恋风辰 | In C++ | 原文链接 (/articlepage?id=2WyQINuJWuRa8CwBM0VOh4kzxkj)

简介

本文介绍C++ 内存模型相关知识，包含几种常见的内存访问策略。

改动序列

在一个C++程序中，每个对象都具有一个改动序列，它由所有线程在对象上的全部写操作构成，其中第一个写操作即为对象的初始化。
大部分情况下，这个序列会随程序的多次运行而发生变化，但是在程序的任意一次运行过程中，所含的全部线程都必须形成相同的改动序列。

改动序列基本要求如下

1 只要某线程看到过某个对象，则该线程的后续读操作必须获得相对新近的值，并且，该线程就同一对象的后续写操作，必然出现在改动序列后方。
2 如果某线程先向一个对象写数据，过后再读取它，那么必须读取前面写的值。
3 若在改动序列中，上述读写操作之间还有别的写操作，则必须读取最后写的值。
4 在程序内部，对于同一个对象，全部线程都必须就其形成相同的改动序列，并且在所有对象上都要求如此.
5 多个对象上的改动序列只是相对关系，线程之间不必达成一致

原子类型

标准原子类型的定义位于头文件 `<atomic>` 内。我们可以通过 `atomic<>` 定义一些原子类型的变量，如 `atomic<bool>` ，`atomic<int>` 这些类型的操作全是原子化的。

从C++17开始，所有的原子类型都包含一个静态常量表达式成员变量， `std::atomic::is_always_lock_free` 。这个成员变量的值表示在任意给定的目标硬件上，原子类型X是否始终以无锁结构形式实现。如果在所有支持该程序运行的硬件上，原子类型X都

以无锁结构形式实现，那么这个成员变量的值就为true；否则为false。

只有一个原子类型不提供 `is_lock_free()` 成员函数： `std::atomic_flag` 。类型 `std::atomic_flag` 的对象在初始化时清零，随后即可通过成员函数 `test_and_set()` 查值并设置成立，或者由 `clear()` 清零。整个过程只有这两个操作。其他的 `atomic<>` 的原子类型都可以基于其实现。

`std::atomic_flag` 的 `test_and_set` 成员函数是一个原子操作，他会先检查 `std::atomic_flag` 当前的状态是否被设置过，

1 如果没被设置过(比如初始状态或者清除后)，将 `std::atomic_flag` 当前的状态设置为 `true` ，并返回 `false` 。

2 如果被设置过则直接返回 `ture` 。

对于 `std::atomic<T>` 类型的原子变量，还支持 `load()` 和 `store()` 、 `exchange()` 、 `compare_exchange_weak()` 和 `compare_exchange_strong()` 等操作。

内存次序

对于原子类型上的每一种操作，我们都可以提供额外的参数，从枚举类std::memory_order取值，用于设定所需的内存次序语义（memory-ordering semantics）。

枚举类std::memory_order具有6个可能的值，

包括 `std::memory_order_relaxed` 、 `std::memory_order_acquire` 、 `std::memory_order_consume` 、

`std::memory_order_acq_rel` 、 `std::memory_order_release` 和 `std::memory_order_seq_cst` 。

存储（ `store` ）操作，可选用的内存次序有 `std::memory_order_relaxed` 、 `std::memory_order_release` 或 `std::memory_order_seq_cst` 。

载入（ `load` ）操作，可选用的内存次序有 `std::memory_order_relaxed` 、 `std::memory_order_consume` 、 `std::memory_order_acquire` 或 `std::memory_order_seq_cst` 。

“读-改-写”（ `read-modify-write` ）操作，可选用的内存次序有 `std::memory_order_relaxed` 、 `std::memory_order_consume` 、 `std::memory_order_acquire` 、 `std::memory_order_release` 、 `std::memory_order_acq_rel` 或 `std::memory_order_seq_cst` 。

原子操作默认使用的是 `std::memory_order_seq_cst` 次序。

这六种内存顺序相互组合可以实现三种顺序模型 (ordering model)

`Sequencial consistent ordering.` 实现同步, 且保证全局顺序一致 (single total order) 的模型. 是一致性最强的模型, 也是默认的顺序模型.
`Acquire-release ordering.` 实现同步, 但不保证保证全局顺序一致的模型.
`Relaxed ordering.` 不能实现同步, 只保证原子性的模型.

实现自旋锁

自旋锁是一种在多线程环境下保护共享资源的同步机制。它的基本思想是，当一个线程尝试获取锁时，如果锁已经被其他线程持有，那么该线程就会不断地循环检查锁的状态，直到成功获取到锁为止。

那我们用这个 `std:atomic_flag` 实现一个自旋锁。

```
#include <iostream>
#include <atomic>
#include <thread>

class SpinLock {
public:
    void lock() {
        //1 处
        while (flag.test_and_set(std::memory_order_acquire)); // 自旋等待，直到成功获取到锁
    }

    void unlock() {
        //2 处
        flag.clear(std::memory_order_release); // 释放锁
    }

private:
    std::atomic_flag flag = ATOMIC_FLAG_INIT;
};
```

我们实现一个测试函数

```
1. void TestSpinLock() {
2.     SpinLock spinlock;
3.     std::thread t1([&spinlock]() {
4.         spinlock.lock();
5.         for (int i = 0; i < 3; i++) {
6.             std::cout << "*";
7.             }
8.         std::cout << std::endl;
9.         spinlock.unlock();
10.        });
11.
12.
13.    std::thread t2([&spinlock]() {
14.        spinlock.lock();
15.        for (int i = 0; i < 3; i++) {
16.            std::cout << "?";
17.        }
18.        std::cout << std::endl;
19.        spinlock.unlock();
20.        });
21.
22.
23.    t1.join();
24.    t2.join();
25. }
```

在主函数执行上述代码会看到如下输出

```
1. ***
2. ???
```

1 处 在多线程调用时，仅有一个线程在同一时刻进入 `test_and_set` ，因为 `atomic_flag` 初始状态为false,所以 `test_and_set` 将 `atomic_flag` 设置为 `true` ，并且返回 `false` 。

比如线程A调用了 `test_and_set` 返回 `false` ，这样 `lock` 函数返回，线程A继续执行加锁区域的逻辑。此时线程B调用 `test_and_set` ， `test_and_set` 会返回true,导致线程B在 `while` 循环中循环等待，达到自旋检测标记的效果。当线程A直行至2处调用 `clear` 操作后， `atomic_flag` 被设置为清空状态，线程B调用 `test_and_set` 会将状态设为成立并返回false，B线程执行加锁区域的逻辑。

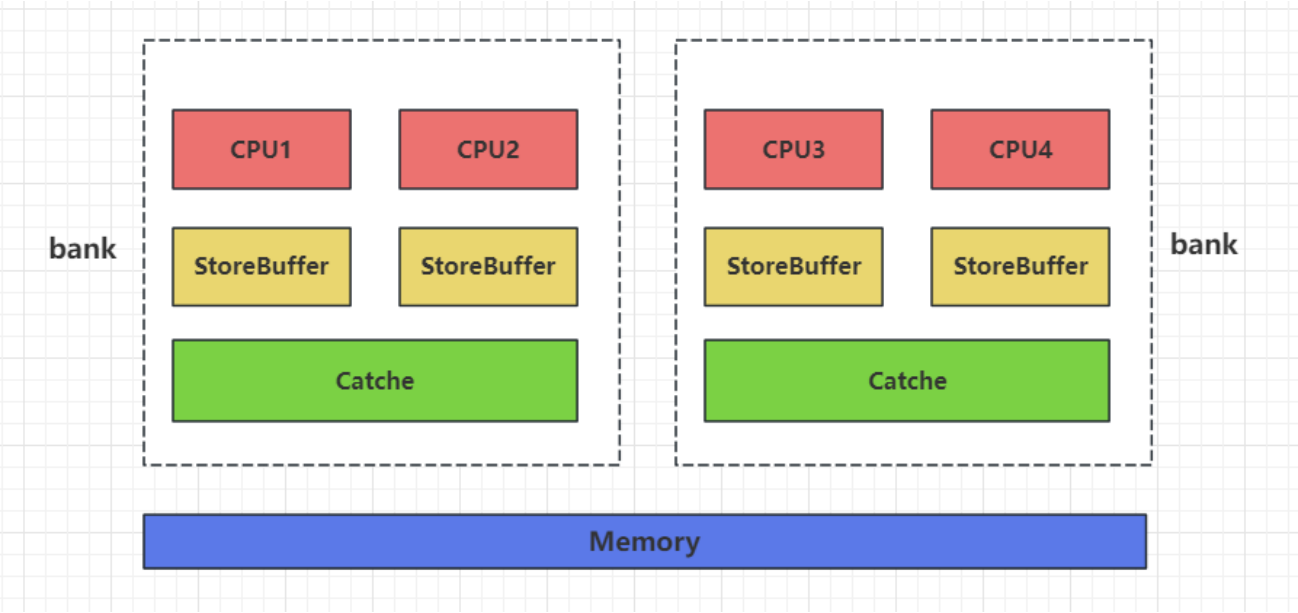
我们看到在设置时使用 `memory_order_acquire` 内存次序，在清除时使用

了 `memory_order_release` 内存次序。

宽松内存序

为了给大家介绍不同的字节序，我们先从最简单的字节序 `std::memory_order_relaxed` (宽松字节序)介绍。
因为字节序是为了实现改动序列的，所以为了理解字节序还要结合改动序列讲起。

我们先看一个CPU和内存结构图



其中StoreBuffer就是一级Cache， Catche是二级Cache，Memory是三级Cache。

每个标识CPU的块就是core，上图画的就是4核结构。每两个core构成一个bank，共享一个cache。四个core共享memory。

每个CPU所作的store均会写到store buffer中，每个CPU会在任何时刻将store buffer中结果写入到cache或者memory中。

那该如何保证数据一致性？这就要提及MESI一致性协议。

MESI 协议，是一种叫作写失效（Write Invalidate）的协议。在写失效协议里，只有一个 CPU 核心负责写入数据，其他的核心，只是同步读取到这个写入。在这个 CPU 核心写入 cache 之后，它会去广播一个“失效”请求告诉所有其他的 CPU 核心。

MESI 协议对应的四个不同的标记，分别是：

M：代表已修改（Modified）

E：代表独占（Exclusive）

S：代表共享（Shared）

I：代表已失效（Invalidated）

“已修改”用来告诉其他cpu已经修改完成，其他cpu可以向cache中写入数据。

“独占”表示数据只是加载到当前 CPU核 的store buffer中，其他的 CPU 核，并没有加载对应的数据到自己的 store buffer 里。

这个时候，如果要向独占的 store buffer 写入数据，我们可以自由地写入数据，而不需要告知其他 CPU 核。

那么对应的，共享状态就是在多核中同时加载了同一份数据。所以在共享状态下想要修改数据要先向所有的其他 CPU 核心广播一个请求，要求先把其他 CPU 核心里面的 cache ，都变成无效的状态，然后再更新当前 cache 里面的数据。

我们可以这么理解，如果变量a此刻在各个cpu的StoreBuffer中，那么CPU1核修改这个a的值，放入cache时通知其他CPU核写失效，因为同一时刻仅有一个CPU核可以写数据，但是其他CPU核是可以读数据的，那么其他核读到的数据可能是CPU1核修改之前的。这就涉及我们提到的改动序列了。

这里给大家简单介绍两个改动序列的术语

1 “ `synchronizes-with` “ : 同步, “A `synchronizes-with` B” 的意思就是 A和B同步，简单来说如果多线程环境下，有一个线程先修改了变量m，我们将这个操作叫做A，之后有另一个线程读取变量m，我们将这个操作叫做B，那么B一定读取A修改m之后的最新值。也可以称作 A “ `happens-before` “ B，即A操作的结果对B操作可见。

2 “ `happens-before` “ : 先行，”A `happens-before` B” 的意思是如果A操作先于B操作，那么A操作的结果对B操作可见。” `happens-before` “包含很多种境况，不仅仅是我们上面提到的” `synchronizes-with` “,之后给大家一个脑图详细说明” `happens-before` “的几种情况。

我们接下来谈谈 `std::memory_order_relaxed` 。

关于 `std::memory_order_relaxed` 具备如下几个功能：

1 作用于原子变量
2 不具有 `synchronizes-with` 关系

3 对于同一个原子变量，在同一个线程中具有 `happens-before` 关系, 在同一线程中不同的原子变量不具有 `happens-before` 关系，可以乱序执行。
4 多线程情况下不具有 `happens-before` 关系。

由上述可知，如果采用最松散的内存顺序模型，在一个线程中，如果某个表达式已经看到原子变量的某个值a，则该表达式的后续表达式只能看到a或者比a更新的值。

我们看下面的代码

```
1. std::atomic<bool> x, y;
2. std::atomic<int> z;
3.
4. void write_x_then_y() {
5.     x.store(true, std::memory_order_relaxed);  // 1
6.     y.store(true, std::memory_order_relaxed);  // 2
7. }
8.
9. void read_y_then_x() {
10.     while (!y.load(std::memory_order_relaxed)) { // 3
11.         std::cout << "y load false" << std::endl;
12.     }
13.
14.     if (x.load(std::memory_order_relaxed)) { //4
15.         ++z;
16.     }
17.
18. }
```

上面的代码封装了两个函数， `write_x_then_y` 负责将 `x` 和 `y` 存储为 true。 `read_y_then_x` 负责读取 `x` 和 `y` 的值。

接下来我们写如下函数调用上面的两个函数

```
1. void TestOrderRelaxed() {
2.     std::thread t1(write_x_then_y);
3.     std::thread t2(read_y_then_x);
4.     t1.join();
5.     t2.join();
6.     assert(z.load() != 0); // 5
7. }
```

上面的代码 `assert` 断言 `z` 不为0，但有时运行到5处 `z` 会等于0触发断言。

我们从两个角度分析

1 从cpu架构分析

假设线程t1运行在CPU1上，t2运行在CPU3上，那么t1对x和y的操作，t2是看不到的。

比如当线程t1运行至1处将x设置为true，t1运行至2处将y设置为true。这些操作仅在CPU1的store buffer中，还未放入cache和memory中，CPU2自然不知道。

如果CPU1先将y放入memory，那么CPU2就会读取y的值为true。那么t2就会运行至3处从while循环退出，进而运行至4处，此时CPU1还未将x的值写入memory，

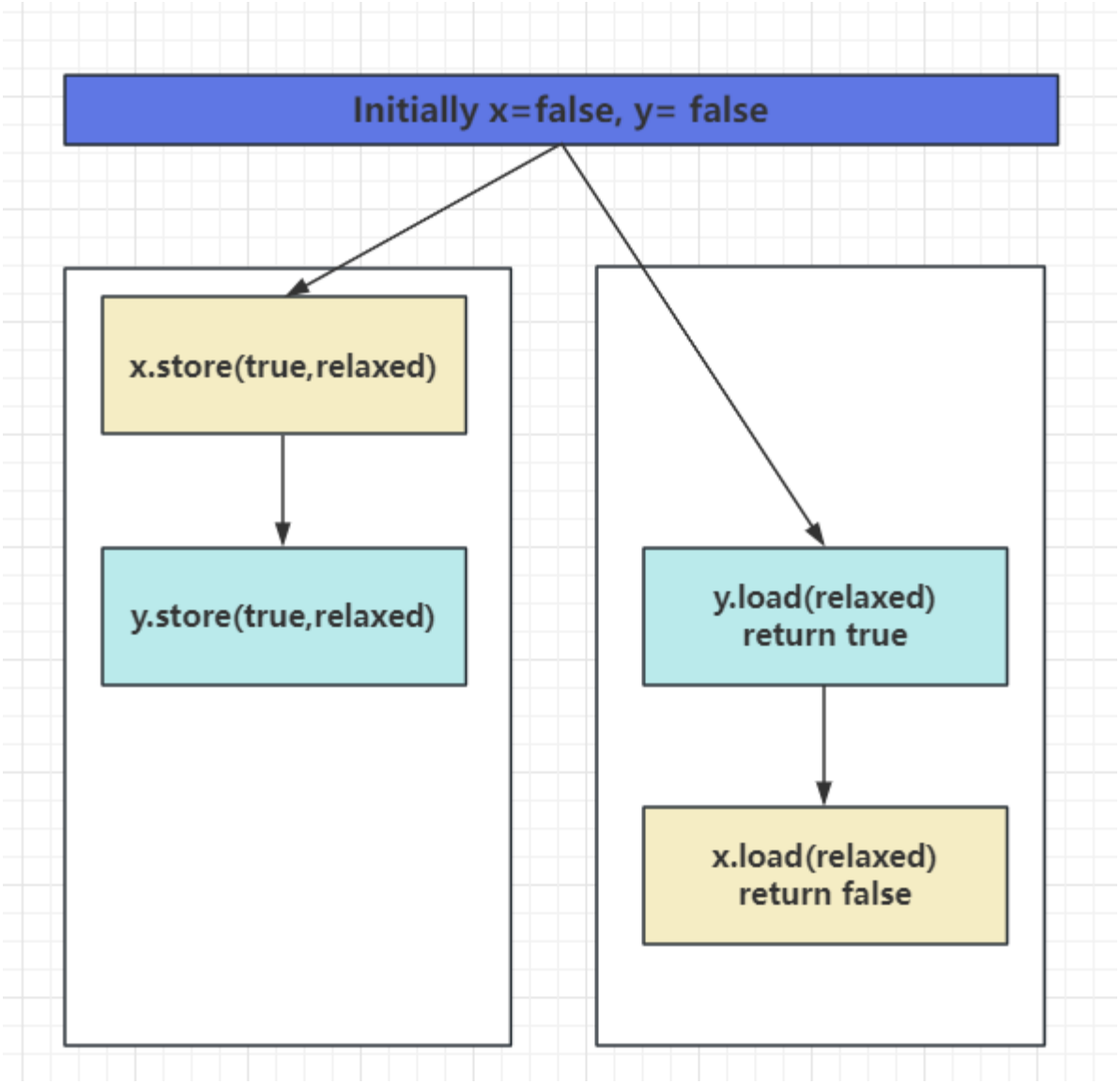
t2读取的x值为false，进而线程t2运行结束，然后CPU1将x写入true, t1结束运行，最后主线程运行至5处，因为z为0,所以触发断言。

2 从宽松内存序分析

因为 `memory_order_relaxed` 是宽松的内存序列，它只保证操作的原子性，并不能保证多个变量之间的顺序性，也不能保证同一个变量在不同线程之间的可见顺序。

比如t1可能先运行2处代码再运行1处代码，因为我们的代码会被编排成指令执行，编译器在不破坏语义的情况下(2处和1处代码无耦合，可调整顺序)，2可能先于1执行。如果这样，t2运行至3处退出while循环，继续运行4处，此时t1还未执行1初代码，则t2运行4处条件不成立不会对z做增加，t2结束。这样也会导致z为0引发断言。

画个图说明上述情况


Initially x=false, y= false
x.store(true,relaxed)
y.store(true,relaxed)
y.load(relaxed)
return true
x.load(relaxed)
return false

我们在看一个例子

```
1. void TestOderRelaxed2() {
2.     std::atomic<int> a{ 0 };
3.     std::vector<int> v3, v4;
4.         std::thread t1([&a]() {
5.             for (int i = 0; i < 10; i += 2) {
6.                 a.store(i, std::memory_order_relaxed);
7.             }
8.         });
9.
10.        std::thread t2([&a]() {
11.            for (int i = 1; i < 10; i += 2)
12.                a.store(i, std::memory_order_relaxed);
13.            });
14.
15.
16.        std::thread t3([&v3, &a]() {
17.            for (int i = 0; i < 10; ++i)
18.                v3.push_back(a.load(std::memory_order_relaxed));
19.            });
20.
21.        std::thread t4([&v4, &a]() {
22.            for (int i = 0; i < 10; ++i)
23.                v4.push_back(a.load(std::memory_order_relaxed));
24.            });
25.
26.        t1.join();
27.        t2.join();
28.        t3.join();
29.        t4.join();
30.
31.        for (int i : v3) {
32.            std::cout << i << " ";
33.        }
34.
35.        std::cout << std::endl;
36.        for (int i : v4) {
37.            std::cout << i << " ";
38.        }
39.        std::cout << std::endl;
40. }
```

线程t1向a中存储偶数,线程t2向a中存储奇数。线程t3从a读取数据写入v3中，线程t4从线程a

中读取数据写入v4中。这四个线程并发执行，最后打印v3和v4的数据。
如果机器性能足够好我们看到的可能是这种输出

```
1. 9 9 9 9 9 9 9 9 9 9
2. 9 9 9 9 9 9 9 9 9 9
```

也可能是这种

```
1. 0 1 7 6 8 9 9 9 9 9
2. 0 2 1 4 5 7 6 8 9 9
```

但我们能确定的是如果v3中7先于6，8，9等，那么v4中也是7先于6，8，9。

因为多个线程仅操作了a变量，通过 `memory_order_relaxed` 的方式仅能保证对a的操作是原子的(同一时刻仅有一个线程写a的值，但是可能多个线程读取a的值)。

但是多个线程之间操作不具备同步关系，也就是线程t1将a改为7，那么线程t3不知道a改动的最新值为7，它读到a的值为1。只是要过一阵子可能会读到7或者a变为7之后又改动的其他值。

但是t3,t4两个线程读取a的次序是一致的，比如t3和t4都读取了7和9，t3读到7在9之前，那么t4也读取到7在9之前。

因为我们 `memory_order_relaxed` 保证了多线程对同一个变量的原子操作的安全性，只是可见性会有延迟罢了。

先行(Happens-before)

`Happens-before` 是一个非常重要的概念. 如前文我们提及:
如果操作 a “ `happens-before` ” 操作 b, 则操作 a 的结果对于操作 b 可见. `happens-before` 的关系可以建立在用一个线程的两个操作之间, 也可以建立在不同的线程的两个操作之间。

顺序先行(sequenced-before)

单线程情况下前面的语句先执行，后面的语句后执行。操作a先于操作b，那么操作b可以看到操作a的结果。我们称操作a顺序先行于操作b。也就是”a sequenced-before b”。

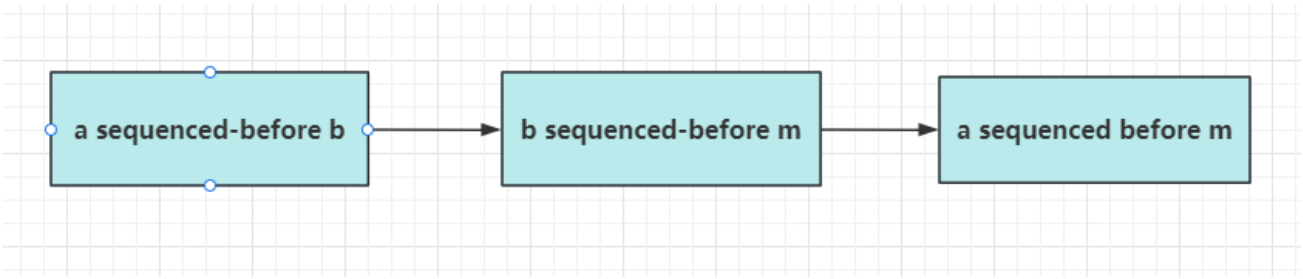
这种情况下”a happens before b”

比如下面

```
1. int main(){
2.     //操作a
3.     int m = 100;
4.     //操作b
5.     std::cout << "m is " << std::endl;
6. }
```

上面操作b 能读取m的值为100.

“sequencde-before”具备传递性，比如操作 a “sequenced-before” 操作 b, 且操作 b “sequenced-before” 操作 m, 则操作 a “sequenced-before” 操作 m.

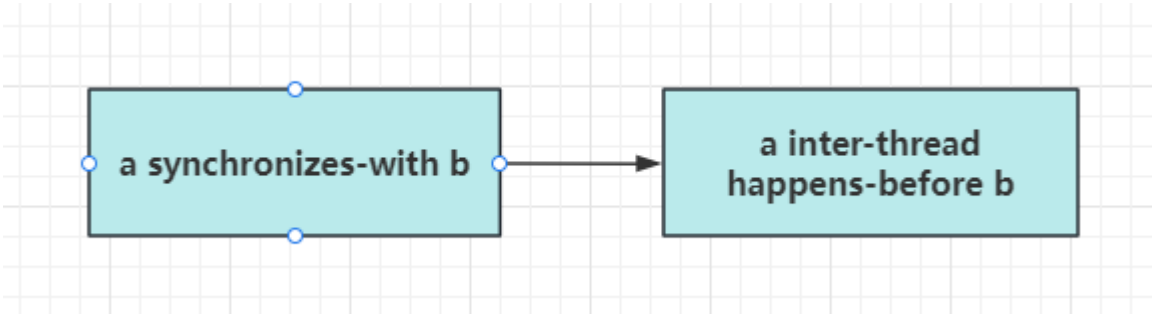


线程间先行

线程间先行又叫做”inter-thread-happens-before”，这是多线程情况的”happens-before”.

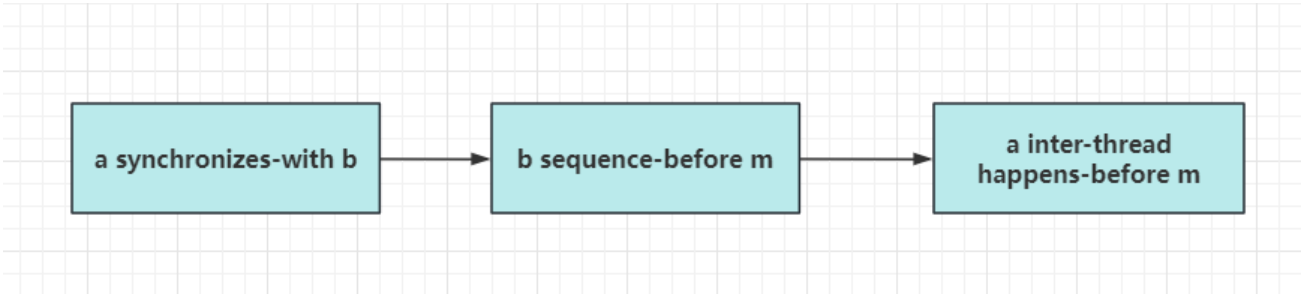
我们前面提到的”synchronizes-with” 可以构成 “happens-before”。

如果线程 1 中的操作 a “synchronizes-with” 线程 2 中的操作 b, 则操作 a “inter-thread happens-before” 操作 b.



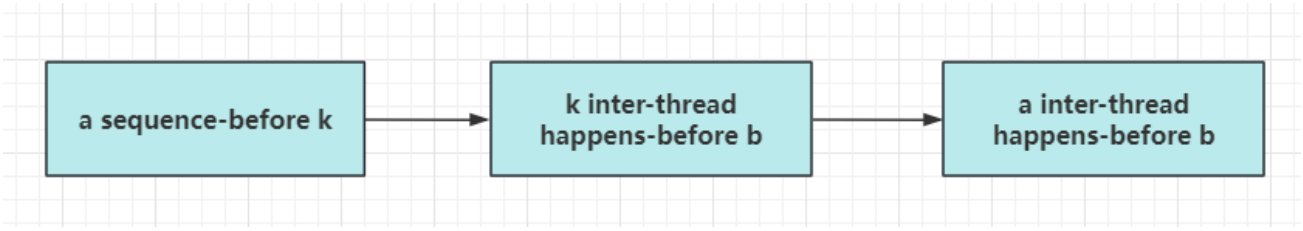
此外 synchronizes-with 还可以 “后接” 一个 sequenced-before 关系组合成 inter-thread happens-before 的关系:

比如操作 a “synchronizes-with” 操作 b, 且操作 b “sequenced-before” 操作 m, 则操作 a “inter-thread happens-before” 操作 m.

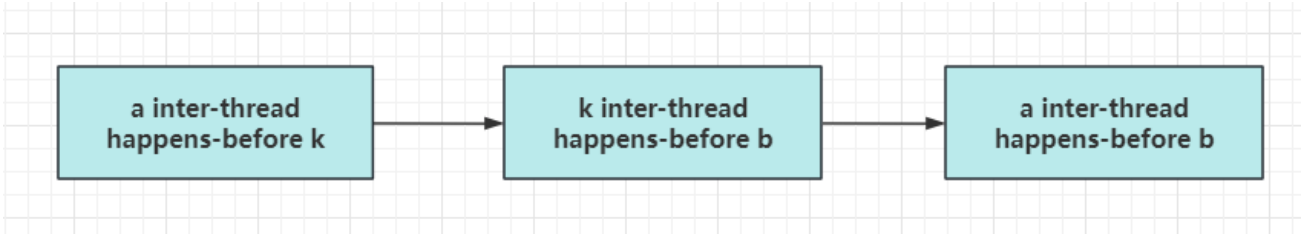


那同样的道理， Inter-thread happens-before 关系则可以 “前接” 一个 sequenced-before 关系以延伸它的范围; 而且 inter-thread happens-before 关系具有传递性:

1 如果操作 a “sequenced-before” 操作 k, 且操作 k “inter-thread happens-before” 操作 b, 则操作 a “inter-thread happens-before” 操作 b.



2 如果操作 a “inter-thread happens-before” 操作 k, 且操作 k “inter-thread happens-before” 操作 b, 则操作 a “inter-thread happens-before” 操作 b.



依赖关系

依赖关系有 carries dependency 和 dependency-ordered before.

单线程情况下a “sequenced-before” b, 且 b 依赖 a 的数据, 则 a “carries a dependency into” b. 称作 a 将依赖关系带给 b, 也理解为b依赖于a。

看下面的代码

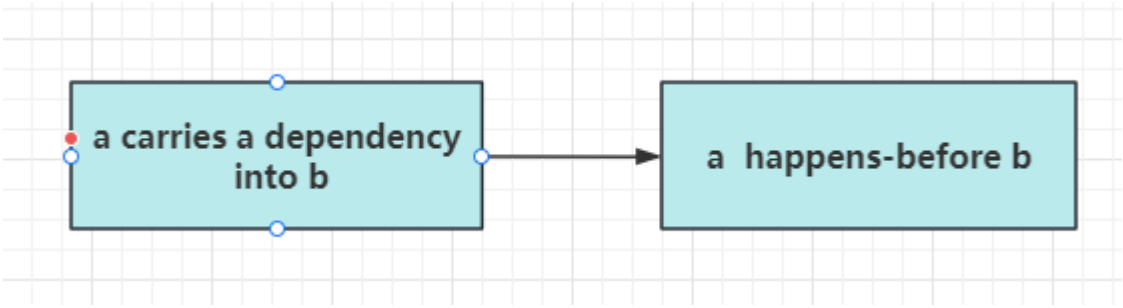
```
1. void TestDependency() {
2.     // 1 处
3.     std::string str = "hello world!";
4.     // 2 处
5.     int i = 3;
6.     // 3 处
7.     std::cout << str[i] << std::endl;
8. }
```

函数 `TestDependency` 内部打印 `str[i]` 的值。3处代码需要依赖1处和2处两个变量的值，所以达成依赖关系。

我们看单线程情况下按顺序执行1，2，3处代码，1 “sequenced-before” 3，且3 依赖 1的数据，则 1 “carries a dependency into” 3

同样的道理 2 “sequenced-before” 3, 且3依赖2 的数据，则2 “carries a dependency into” 3.

“carries a dependency into” 也被归为”happens-before”。



2 多线程情况下

线程1执行操作A(比如对i自增)，线程2执行操作B(比如根据i访问字符串下表的元素), 如果线程1先于线程2执行，且操作A的结果对操作B可见，我们将这种叫做
A “dependency-ordered before” B. 有人会说这不是前面说到的A “synchronizes with “ B吗? 你可以这么理解。就当作他们达到的效果是一致的，只不过A “dependency-ordered before” B 更细化一点，表述了一种依赖，比如操作A仅仅对i增加，而没有对字符串修改。而操作B需要通过i访问字符串数据。那操作B实际上是依赖于A的。

Happens-before不代表指令执行顺序

Happens-before不代表指令实际执行顺序，C++编译器可以对不相关的指令任意编排达到优化效果，Happens-before仅是C++语义层面的描述，表示 a “Happens-before” b仅能说明a操

作的结果对b操作可见。

看这样一段代码

```
1. int  Add() {
2.     int a = 0, b = 0;
3.     //1 处
4.     a++;
5.     // 2 处
6.     b++;
7.     // 3 处
8.     return  a + b;
9. }
```

单线程执行上述代码，操作1一定是happens-before 操作2 的(a “sequenced-before” b)，就是我们理解的 `a++` 先于 `b++` 。

但是计算机的指令可能不是这样，一条C++语句对于多条计算机指令。

有可能是先将b值放入寄存器eax做加1，再将a的值放入寄存器edx做加1，然后再将eax寄存器的值写回a，将edx写回b。

因为对于计算机来说1处操作和2处操作的顺序对于3处来说并无影响。只要3处返回 `a+b` 之前能保证a和b的值是增加过的即可。

那我们语义上的”Happens-before”有意义吗？ 是有意义的，因为如果 a “sequenced-before” b, 那么无论指令如何编排，最终写入内存的顺序一定是a先于b。

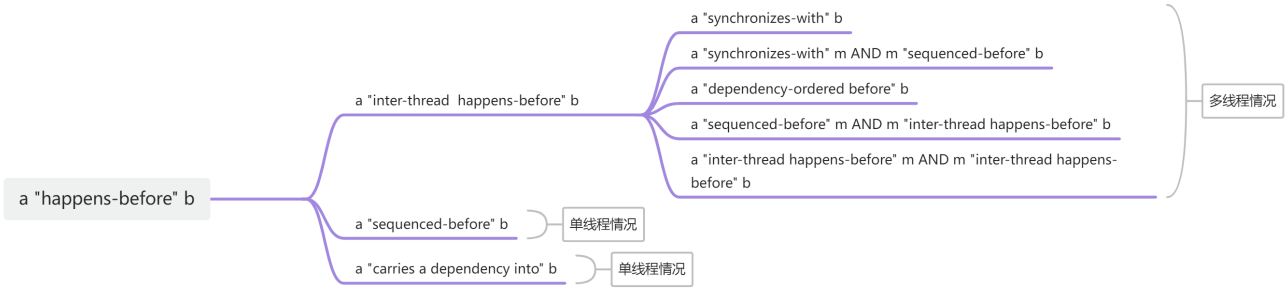
只不过C++编译器不断优化尽可能不造成指令编排和语义理解的差异，上面C++的代码转换为汇编指令如下

```
1.    int a = 0, b = 0;
2. 00A1C8F5  mov        dword ptr [a],0
3. 00A1C8FC  mov        dword ptr [b],0
4.    //1 处
5.    a++;
6. 00A1C903  mov        eax,dword ptr [a]
7. 00A1C906  add        eax,1
8. 00A1C909  mov        dword ptr [a],eax
9.    // 2 处
10.   b++;
11. 00A1C90C  mov        eax,dword ptr [b]
12. 00A1C90F  add        eax,1
13. 00A1C912  mov        dword ptr [b],eax
14.   return  a + b;
15. 00A1C915  mov        eax,dword ptr [a]
16. 00A1C918  add        eax,dword ptr [b]
```

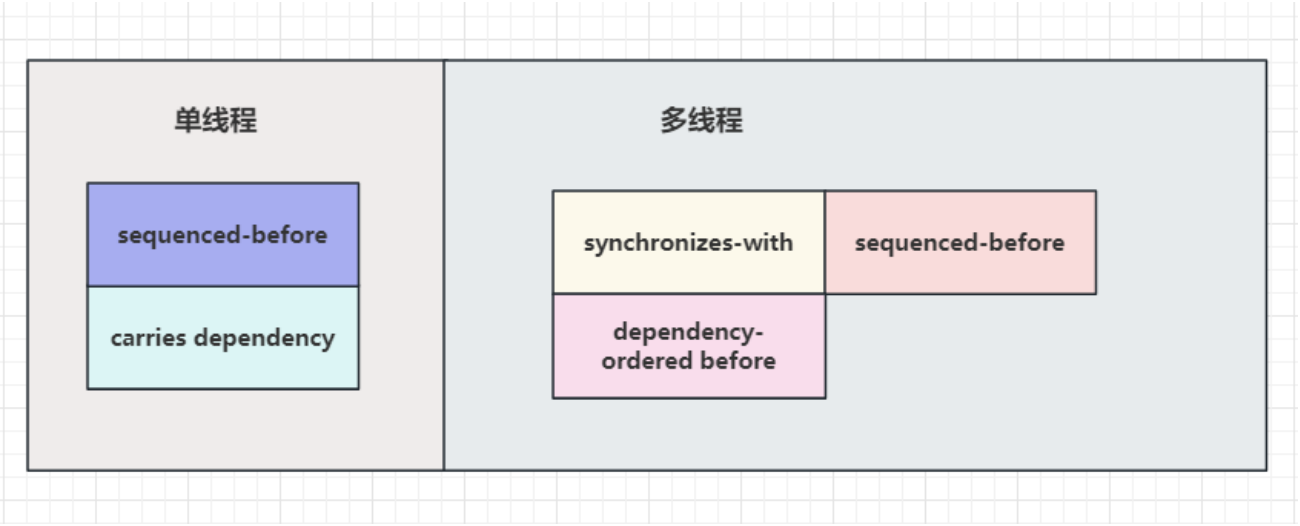
可以看到C++编译器尽可能不造成语义理解和指令编排上的歧义。

脑图

我们将”happens-before” 的几种情况做成脑图，方便理解



我们画一个框将”happens-before” 的几种情况框起来



总结

本文介绍了3种内存模型，包括全局一致性模型，同步模型以及最宽松的原子模型，以及6种内存序，下一篇将介绍如何利用6中内存序达到三种模型的效果。

详细源码

https://gitee.com/secondtonone1/boostasio-learn/tree/master/concurrent/day10-MemoryModel (https://gitee.com/secondtonone1/boostasio-learn/tree/master/concurrent/day10-MemoryModel)

视频链接

https://space.bilibili.com/271469206/channel/collectiondetail?sid=1623290 (https://space.bilibili.com/271469206/channel/collectiondetail?sid=1623290)

浏览(2337) (JavaScript:void(0);) 评论(0) (JavaScript:void(0);) 喜欢(553) (JavaScript:void(0);)

热门评论

请输入正文

评论

没有更多评论了,点击收起 ≪ (JavaScript:void(0);)

热门文章

slice介绍和使用

♡喜欢(521) 浏览(1546)

C++ 类的继承封装和多态

♡喜欢(588) 浏览(2196)

windows环境搭建和vscode配置

♡喜欢(587) 浏览(1595)

解密定时器的实现细节

♡喜欢(566) 浏览(1449)

Linux环境搭建和编码

♡喜欢(594) 浏览(5076)

最新评论

C++ 类的继承封装和多态

gzy:谢谢老师，写的真的很好，本来对这些新的关键字都很模糊，几篇下来配合着ai记着笔记一下子就清晰了

C++ 右值引用与移动构造函数

李程浩:1

C++ 线程管控

secondtonone1:好的，已修复，写的时候马虎了，谢谢指正。

线程基础

secondtonone1:谢谢，笔误已修复

再谈单例模式

secondtonone1:是的，C++11以后返回局部static变量对象能保证线程安全了。

线程安全的无锁栈

secondtonone1:谢谢支持，如果pop的次数大于push的次数是会让线程处于重试的，这个是测试用例，必须满足push和pop的次数相同，实际情况不会这么使用。栈的设计没有问题。

创建项目和编译

secondtonone1:谢谢支持

visual studio配置boost库

secondtonone1:是的，您说的是对的，windows平台要采用msvc方式。

C++ 类的拷贝构造、赋值运算、单例模式

secondtonone1:好的，已修复。

asio多线程模式IOThreadPool

secondtonone1:这么优秀吗

asio实现http服务器

secondtonone1:我认为IO复用节省了线程资源，不用像传统服务器一样为每个连接单独创立线程，而是统一通过epoll或者iocp通知。

类和对象

陈宇航:支持！！！！

解决博客回复区被脚本注入的问题

secondtonone1:走到现在我忽然明白一个道理，无论工作也好生活也罢，最重要的是开心，即使一份安稳的工作不能给我带来事业上的积累也要合理的舍弃，所以我还是想去做喜欢的方向。

泛型算法的定制操作

secondtonone1:lambda和bind是C11新增的利器，善于利用这两个机制可以极大地提升编程安全性和效率。

双链表实现LRU算法

secondtonone1:双链表插入和删除节点是本篇的难点，多多练习即可。

构造函数

secondtonone1:构造函数是类的基础知识，要着重掌握

面试题汇总(一)

secondtonone1:看到网络上经常提问的go的问题，做了一下汇总，结合自己的经验给出的答案，如有纰漏，望指正批评。

使用mongo

secondtonone1:mongo操作是程序员必备的基础技能，包括增删改查，以及较为复杂的查询，分组查询，条件查询，模糊查询，文档内查询等，本文结合自己实际开发经验做了整理。

interface应用

secondtonone1:interface是万能类型，但是使用时要转换为实际类型来使用。interface丰富了go的多态特性，也降低了传统面向对象语言的耦合性。

golang 函数介绍

secondtonone1:函数是go中的一等公民，作为新兴语言，go摒弃了面向对象的一些糟粕，采取接口方式编程，而接口方式编程都是基于函数的，参数为interface，进而达到泛型作用，比如sort排序，只需要传入的参数满…

堆排序

secondtonone1:堆排序非常实用，定时器就是这个原理制作的。

Linux环境搭建和编码

恋恋风辰:Linux环境下go的安装比较简单，可以不用设置GOPATH环境变量，后期我们学习go mod 之后就拜托了go文件目录的限制了。

slice介绍和使用

恋恋风辰:切片作为引用类型极大的提高了数据传递的效率和性能，但也要注意切片的浅拷贝隐患，算是一把双刃剑，这世间的常态就是在两极之间寻求一种稳定。

其他链接

GitHub (https://github.com)
个人博客 (https://www.limerence2017.com)
博客园 (https://www.cnblogs.com/secondtonone1/)
CSDN (https://blog.csdn.net/secondtonone1)

B站



个人微信



恋恋风辰

北京 朝阳



扫一扫上面的二维码图案，加我为朋友。

个人公众号



本站累计访问量204866次

本网站内容全部为原创，谢绝转载。

京公网安备 11010502047625号 (http://www.beian.gov.cn/portal/registerSystemInfo?recordcode=11010502047625)

京ICP备2021033081号 (https://beian.miit.gov.cn/)